

Student Management System

Name: *Aditya Kumar*

Technology: *Java, Spring Boot, MongoDB, Postman*

STUDENT MANAGEMENT SYSTEM

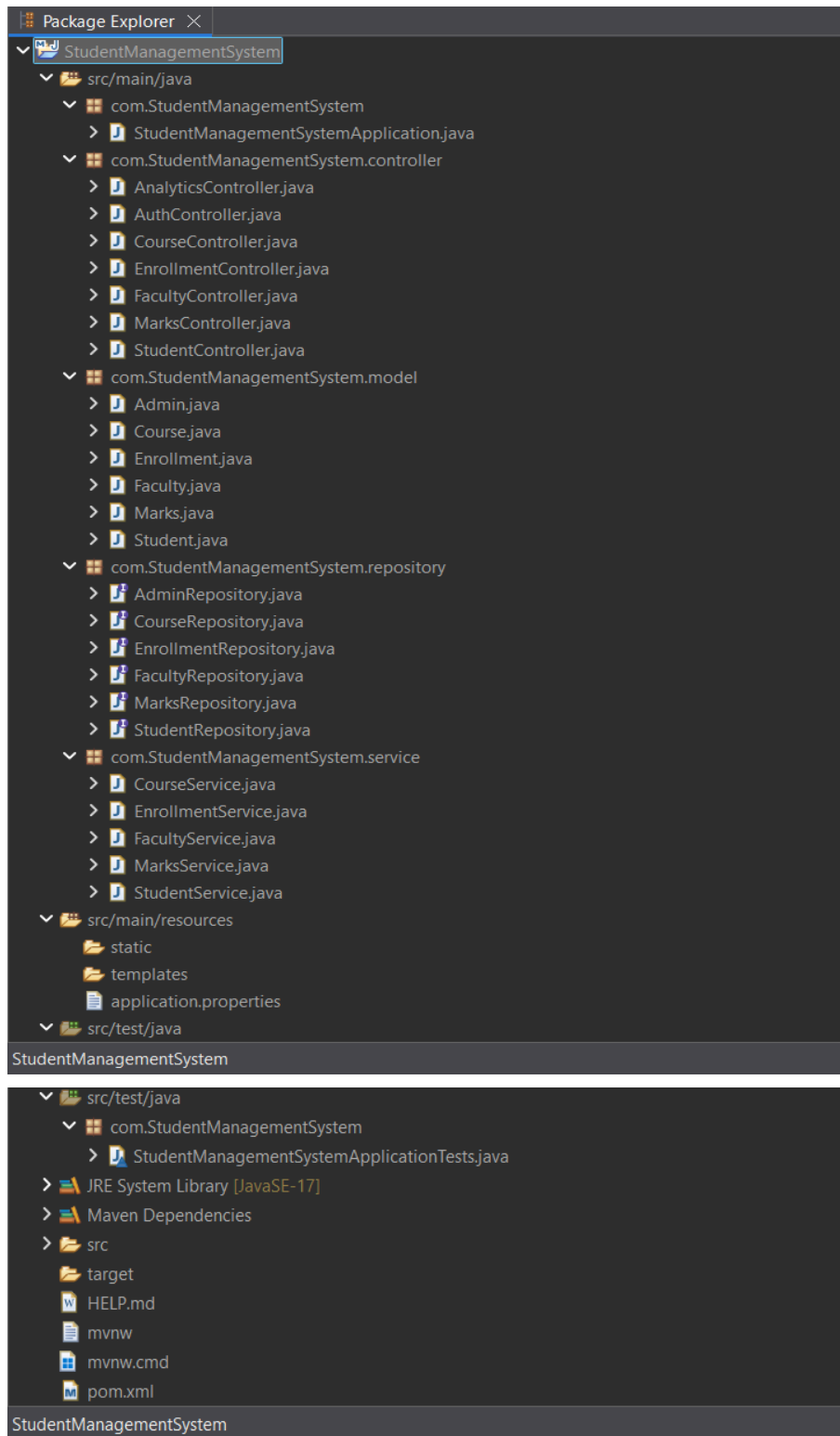
1. ABSTRACT

The Student Management System is a web-based application developed using Spring Boot and MongoDB. The system is designed to manage student-related information efficiently. It provides functionalities such as student registration, course management, faculty management, enrollment management, marks management, attendance management, and user authentication. The application exposes REST APIs that can be tested using Postman and stores data in MongoDB. The project demonstrates CRUD operations, database connectivity, and RESTful web service development.

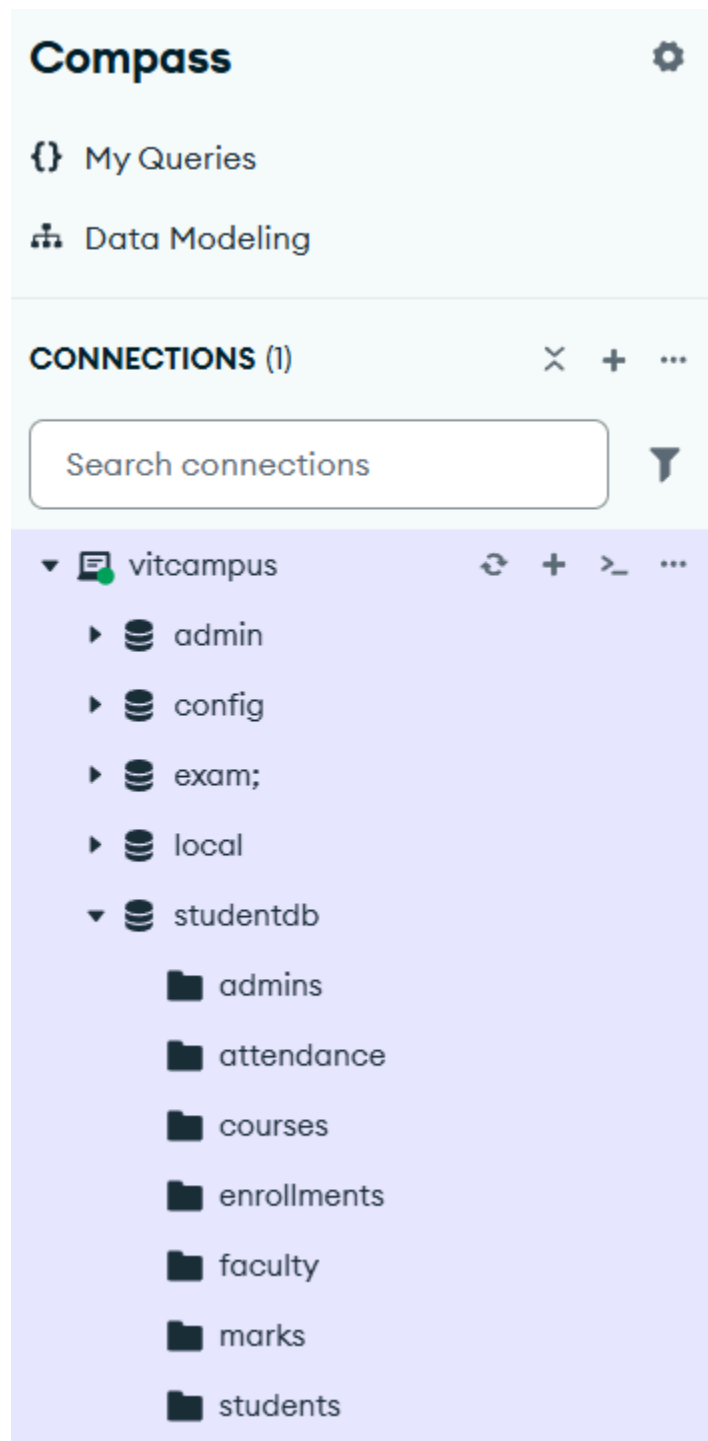
2. OBJECTIVES

- To manage student records efficiently.
- To perform CRUD (Create, Read, Update, Delete) operations.
- To store and retrieve data using MongoDB.
- To implement REST APIs using Spring Boot.
- To manage courses, faculty, enrollments, marks, and attendance.
- To provide authentication for system access.
- To test APIs using Postman

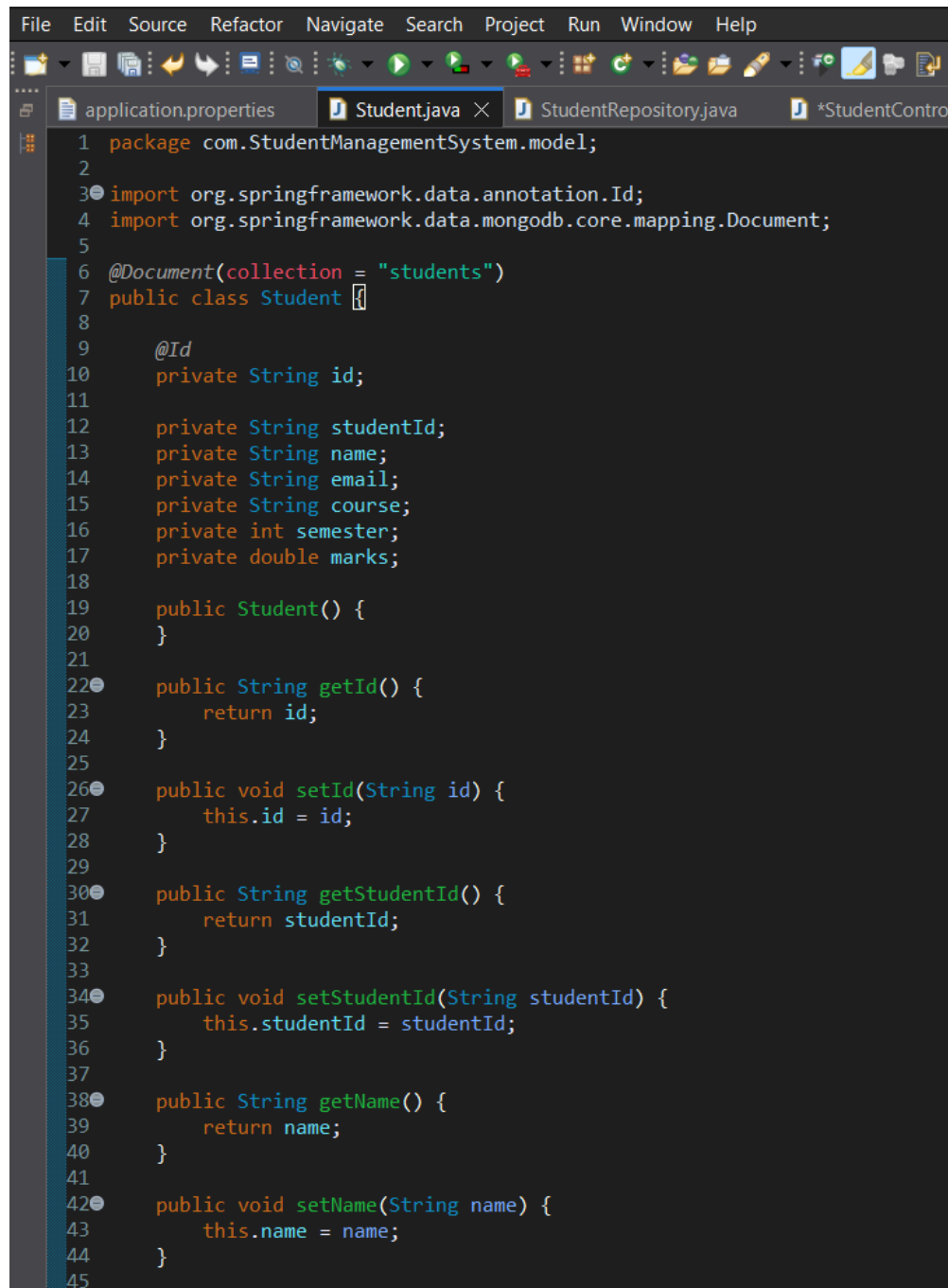
Project Structure (Eclipse)



Mongodb



Student.java Code



The screenshot shows an IDE window with the following tabs: application.properties, Student.java (active), StudentRepository.java, and *StudentContro. The code in Student.java is as follows:

```
1 package com.StudentManagementSystem.model;
2
3 import org.springframework.data.annotation.Id;
4 import org.springframework.data.mongodb.core.mapping.Document;
5
6 @Document(collection = "students")
7 public class Student {
8
9     @Id
10    private String id;
11
12    private String studentId;
13    private String name;
14    private String email;
15    private String course;
16    private int semester;
17    private double marks;
18
19    public Student() {
20    }
21
22    public String getId() {
23        return id;
24    }
25
26    public void setId(String id) {
27        this.id = id;
28    }
29
30    public String getStudentId() {
31        return studentId;
32    }
33
34    public void setStudentId(String studentId) {
35        this.studentId = studentId;
36    }
37
38    public String getName() {
39        return name;
40    }
41
42    public void setName(String name) {
43        this.name = name;
44    }
45}
```

```
45
46 public String getEmail() {
47     return email;
48 }
49
50 public void setEmail(String email) {
51     this.email = email;
52 }
53
54 public String getCourse() {
55     return course;
56 }
57
58 public void setCourse(String course) {
59     this.course = course;
60 }
61
62 public int getSemester() {
63     return semester;
64 }
65
66 public void setSemester(int semester) {
67     this.semester = semester;
68 }
69
70 public double getMarks() {
71     return marks;
72 }
73
74 public void setMarks(double marks) {
75     this.marks = marks;
76 }
77
```

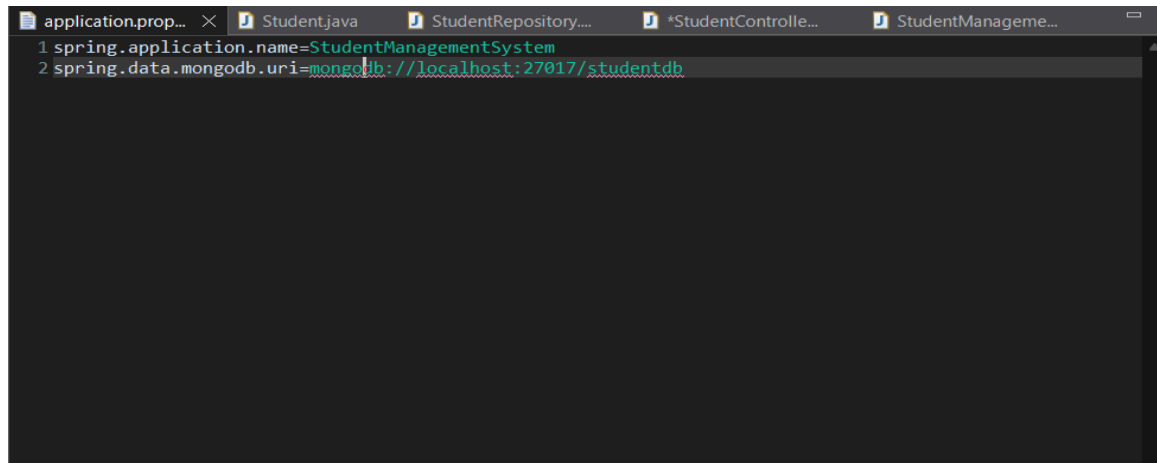
StudentRepository.java

```
1 package com.StudentManagementSystem.repository;
2
3 import org.springframework.data.mongodb.repository.MongoRepository;
4 import com.StudentManagementSystem.model.Student;
5
6 public interface StudentRepository extends MongoRepository<Student, String> {
7
8 }
```

StudentController.java

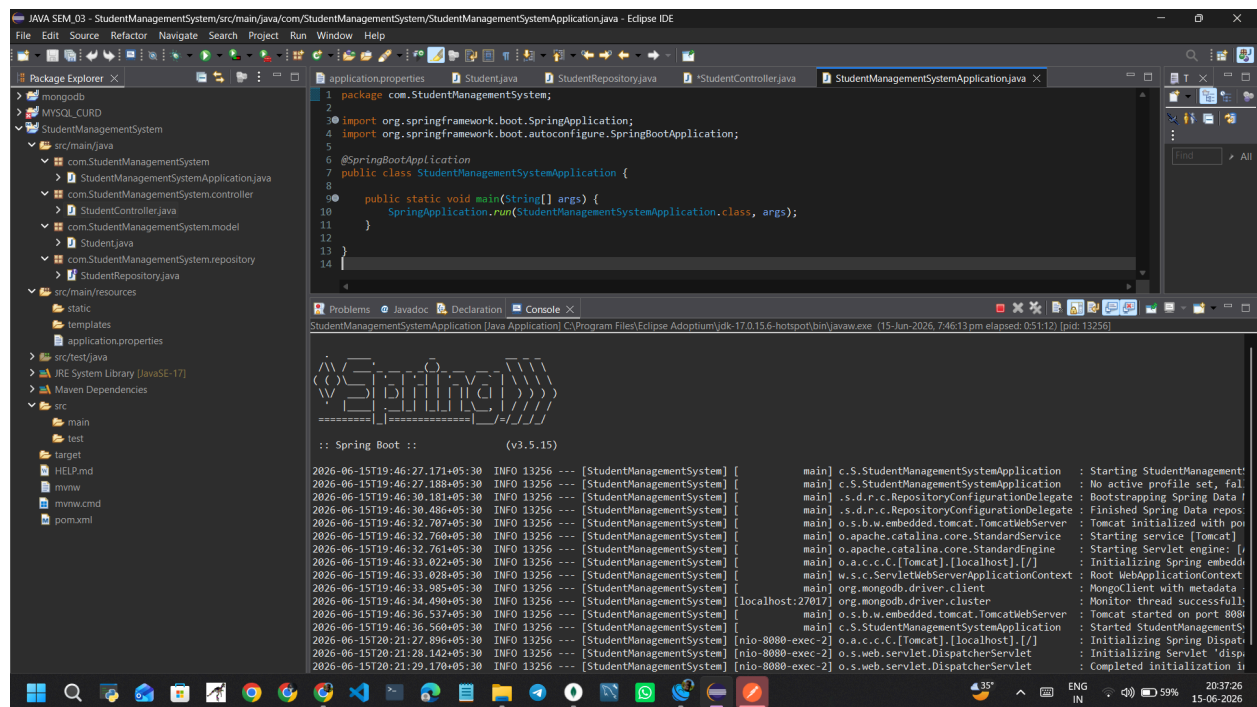
```
1 package com.StudentManagementSystem.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.web.bind.annotation.*;
5
6 import com.StudentManagementSystem.model.Student;
7 import com.StudentManagementSystem.repository.StudentRepository;
8
9 @RestController
10 public class StudentController {
11
12     @Autowired
13     private StudentRepository repo;
14
15     @PostMapping("/students")
16     public Student addStudent(@RequestBody Student student) {
17         return repo.save(student);
18     }
19 }
20
```

Application.properties

A screenshot of an IDE window showing the 'application.properties' file. The window has several tabs open: 'application.prop...', 'Student.java', 'StudentRepository...', '*StudentControlle...', and 'StudentManageme...'. The 'application.properties' file contains two lines of configuration: '1 spring.application.name=StudentManagementSystem' and '2 spring.data.mongodb.uri=mongodb://localhost:27017/studentdb'. The text is color-coded: 'spring' is blue, 'application.name' is black, 'StudentManagementSystem' is green, 'spring.data.mongodb.uri' is black, and 'mongodb://localhost:27017/studentdb' is green. The background of the IDE is dark gray.

```
1 spring.application.name=StudentManagementSystem
2 spring.data.mongodb.uri=mongodb://localhost:27017/studentdb
```


Spring Boot Running



```
1 package com.StudentManagementSystem;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class StudentManagementSystemApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(StudentManagementSystemApplication.class, args);
11     }
12 }
13
14
```

Spring Boot :: (v3.5.15)

```
2026-06-15T19:46:27.171+05:30 INFO 13256 --- [StudentManagementSystem] [main] c.S.StudentManagementSystemApplication : Starting StudentManagementSystemApplication
2026-06-15T19:46:27.188+05:30 INFO 13256 --- [StudentManagementSystem] [main] c.S.StudentManagementSystemApplication : No active profile set, falling back to default profiles: default
2026-06-15T19:46:30.181+05:30 INFO 13256 --- [StudentManagementSystem] [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repository configuration
2026-06-15T19:46:30.486+05:30 INFO 13256 --- [StudentManagementSystem] [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository configuration
2026-06-15T19:46:32.707+05:30 INFO 13256 --- [StudentManagementSystem] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080
2026-06-15T19:46:32.760+05:30 INFO 13256 --- [StudentManagementSystem] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache/2.4.52 (Ubuntu)]
2026-06-15T19:46:33.022+05:30 INFO 13256 --- [StudentManagementSystem] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-15T19:46:33.028+05:30 INFO 13256 --- [StudentManagementSystem] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed
2026-06-15T19:46:33.985+05:30 INFO 13256 --- [StudentManagementSystem] [main] org.mongodb.driver.client : MongoClient with metadata
2026-06-15T19:46:34.490+05:30 INFO 13256 --- [StudentManagementSystem] [localhost:27017] org.mongodb.driver.cluster : Monitor thread successfully started
2026-06-15T19:46:36.537+05:30 INFO 13256 --- [StudentManagementSystem] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s) 8080
2026-06-15T20:21:27.896+05:30 INFO 13256 --- [StudentManagementSystem] [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet
2026-06-15T20:21:28.142+05:30 INFO 13256 --- [StudentManagementSystem] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-15T20:21:29.170+05:30 INFO 13256 --- [StudentManagementSystem] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization
```

Postman Request and response

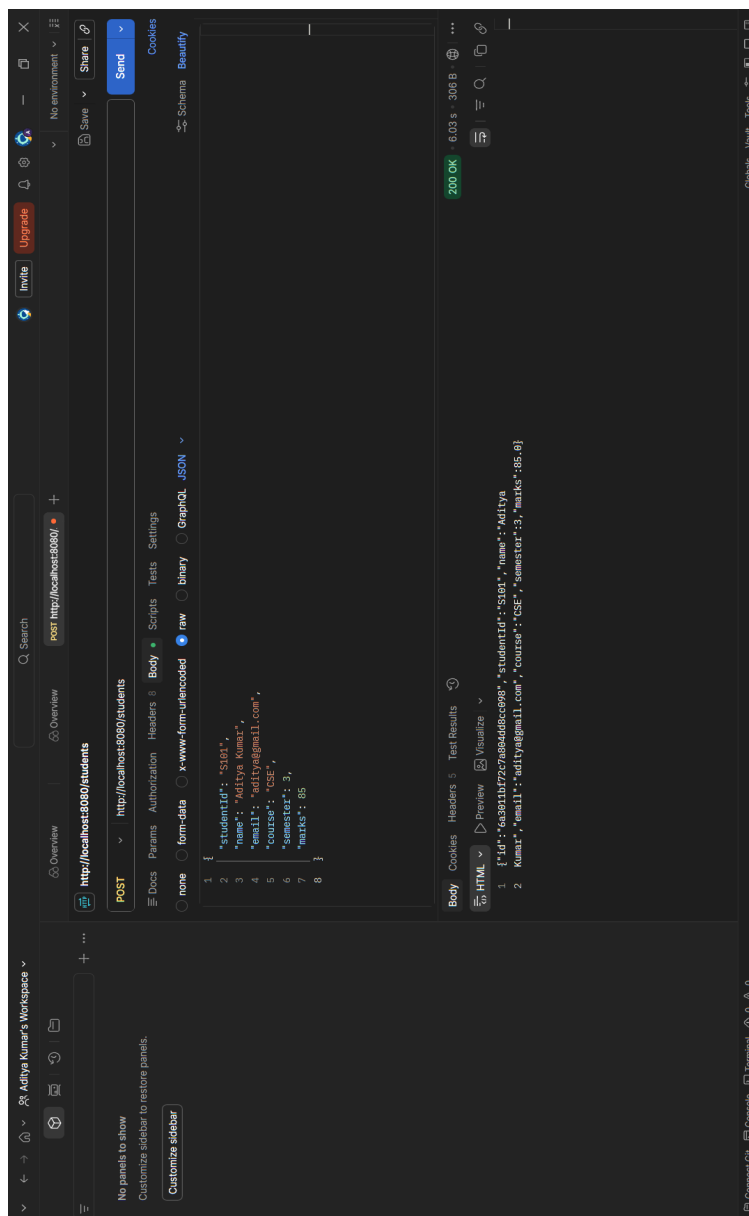
1. CREATE STUDENT

Method: POST

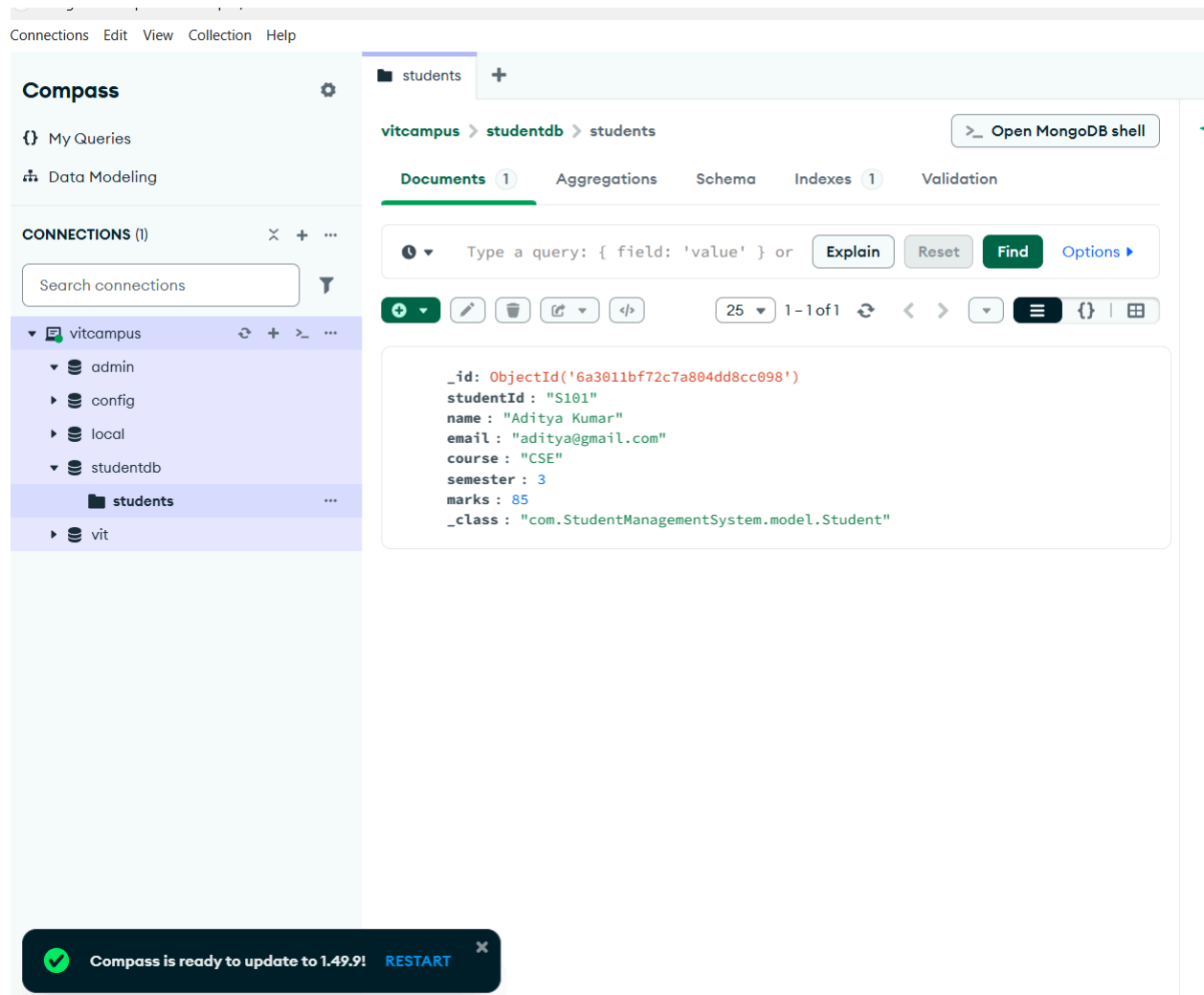
This API is used to add a new student record to the database. The user provides student details in JSON format, and the application stores the information in the MongoDB collection.

URL:

<http://localhost:8080/students>



MongoDB Compass



2. READ ALL STUDENTS

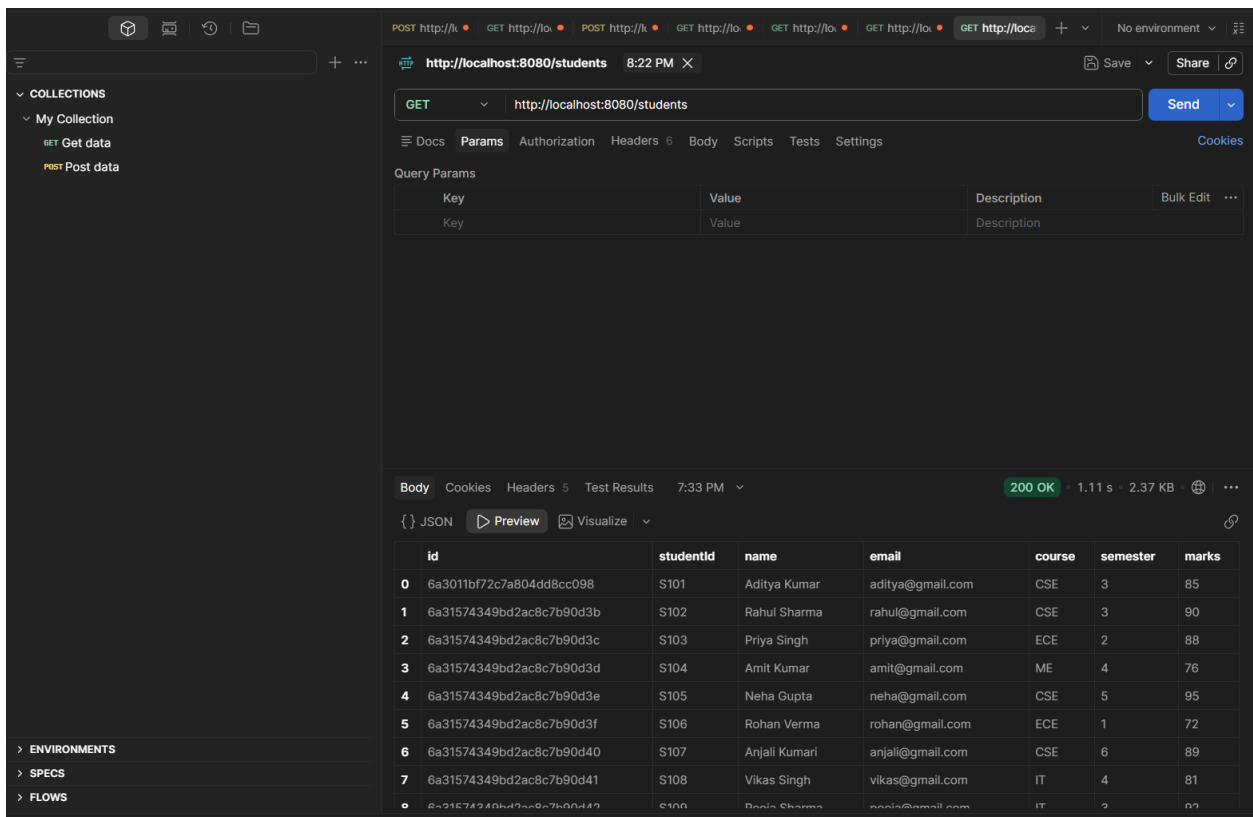
Method: GET

This API retrieves all student records available in the database. It helps administrators view and manage student information efficiently.

URL:

<http://localhost:8080/students>

Result:



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/students
- Status:** 200 OK
- Response Time:** 1.11 s
- Response Size:** 2.37 KB

The response body is a JSON array of 10 student records. The first 9 records are visible in the table below:

	id	studentid	name	email	course	semester	marks
0	6a3011bf72c7a804dd8cc098	S101	Aditya Kumar	aditya@gmail.com	CSE	3	85
1	6a31574349bd2ac8c7b90d3b	S102	Rahul Sharma	rahul@gmail.com	CSE	3	90
2	6a31574349bd2ac8c7b90d3c	S103	Priya Singh	priya@gmail.com	ECE	2	88
3	6a31574349bd2ac8c7b90d3d	S104	Amit Kumar	amit@gmail.com	ME	4	76
4	6a31574349bd2ac8c7b90d3e	S105	Neha Gupta	neha@gmail.com	CSE	5	95
5	6a31574349bd2ac8c7b90d3f	S106	Rohan Verma	rohan@gmail.com	ECE	1	72
6	6a31574349bd2ac8c7b90d40	S107	Anjali Kumari	anjali@gmail.com	CSE	6	89
7	6a31574349bd2ac8c7b90d41	S108	Vikas Singh	vikas@gmail.com	IT	4	81
8	6a31574349bd2ac8c7b90d42	S109	Deepa Sharma	deepa@gmail.com	IT	2	87

3. READ STUDENT BY ID

Method: GET

URL:

<http://localhost:8080/students/S105>

Result:

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/students/S105`
- Method:** GET
- Status:** 200 OK
- Response Time:** 403 ms
- Response Size:** 302 B
- Response Body (JSON):**

```
{  "id": "6a31574349bd2ac8c7b90d3e",  "studentId": "S105",  "name": "Neha Gupta",  "email": "neha@gmail.com",  "course": "CSE",  "semester": 5,  "marks": 95.0}
```

4. UPDATE STUDENT

Method: PUT

This API allows modification of an existing student record. Updated information is saved directly into the MongoDB database, ensuring data remains accurate and up to date.

URL:

<http://localhost:8080/students/S102>

Result:

The screenshot displays an HTTP client interface with a PUT request to `http://localhost:8080/students/S102`. The request body is a JSON object with the following fields: `name`, `email`, `course`, `semester`, and `marks`. The response status is `200 OK` with a response time of `466 ms` and a size of `321 B`. The response body is a JSON object containing the updated student record, including an `id` and `studentId`.

```
1 {
2   "name": "Rahul Sharma Updated",
3   "email": "rahul_updated@gmail.com",
4   "course": "CSE",
5   "semester": 4,
6   "marks": 95
7 }
8
```

```
1 {
2   "id": "6a31574349bd2ac8c7b90d3b",
3   "studentId": "S102",
4   "name": "Rahul Sharma Updated",
5   "email": "rahul_updated@gmail.com",
6   "course": "CSE",
7   "semester": 4,
8   "marks": 95.0
9 }
```

5. DELETE STUDENT

Method: DELETE

This API removes a student record from the database using the student's unique identifier. It demonstrates the delete operation of CRUD functionality.

URL:

<http://localhost:8080/students/S117>

Result:

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/students/S117`
- Method:** `DELETE`
- Send Button:** A blue button labeled "Send".
- Query Params:** A table with columns: Key, Value, Description, Bulk Edit, and a menu icon.
- Body:** A tab labeled "Body" is selected, showing a JSON response.
- Status:** A red banner indicates a "500 Internal Server Error" with a response time of 146 ms and a size of 268 B.
- JSON Response:**

```
1 {  
2   "timestamp": "2026-06-16T14:59:15.852+00:00",  
3   "status": 500,  
4   "error": "Internal Server Error",  
5   "path": "/students/S117"  
6 }
```

6. BULK INSERT STUDENTS

Method: POST

The bulk insert feature enables multiple student records to be added at once. This reduces manual effort and improves efficiency when handling large amounts of student data.

URL:

<http://localhost:8080/students/bulk>

Result:

Multiple student records inserted into MongoDB at once.

The screenshot shows an HTTP client interface with the following details:

- URL:** `http://localhost:8080/students/bulk`
- Method:** `POST`
- Body (raw):**

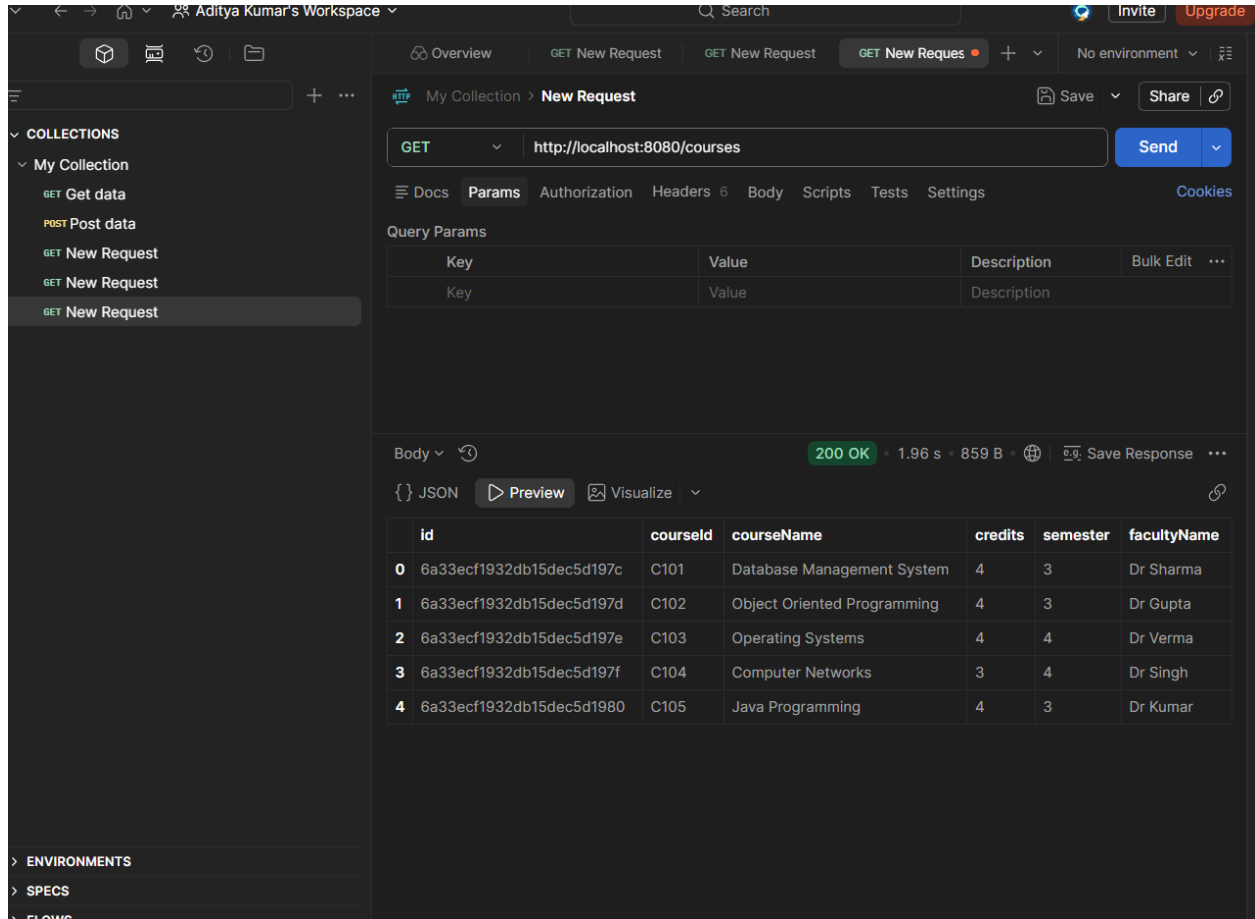
```
102   "course": "IT",
103   "semester": 6,
104   "marks": 84
105 },
106 {
107   "studentId": "S115",
108   "name": "Ritika Jain",
109   "email": "ritika@gmail.com",
110   "course": "ECE",
111   "semester": 2,
112   "marks": 91
113 },
114 {
115   "studentId": "S116",
116   "name": "Abhishek Singh",
```
- Response:** `200 OK` (5.31 s, 2.23 KB)
- Body (JSON):** A table with 11 student records.

ID	Student ID	Name	Email	Course	Semester	Marks	
0	6a31574349bd2ac8c7b90d3b	S102	Rahul Sharma	rahul@gmail.com	CSE	3	90
1	6a31574349bd2ac8c7b90d3c	S103	Priya Singh	priya@gmail.com	ECE	2	88
2	6a31574349bd2ac8c7b90d3d	S104	Amit Kumar	amit@gmail.com	ME	4	76
3	6a31574349bd2ac8c7b90d3e	S105	Neha Gupta	neha@gmail.com	CSE	5	95
4	6a31574349bd2ac8c7b90d3f	S106	Rohan Verma	rohan@gmail.com	ECE	1	72
5	6a31574349bd2ac8c7b90d40	S107	Anjali Kumari	anjali@gmail.com	CSE	6	89
6	6a31574349bd2ac8c7b90d41	S108	Vikas Singh	vikas@gmail.com	IT	4	81
7	6a31574349bd2ac8c7b90d42	S109	Pooja Sharma	pooja@gmail.com	IT	3	92
8	6a31574349bd2ac8c7b90d43	S110	Karan Patel	karan@gmail.com	CSE	2	68
9	6a31574349bd2ac8c7b90d44	S111	Sneha Roy	sneha@gmail.com	ECE	5	87

Additional Module Screenshots

Course Module

The Course Module stores and manages information about various courses offered by the institution. It helps maintain course details such as course ID, name, semester, and credits.

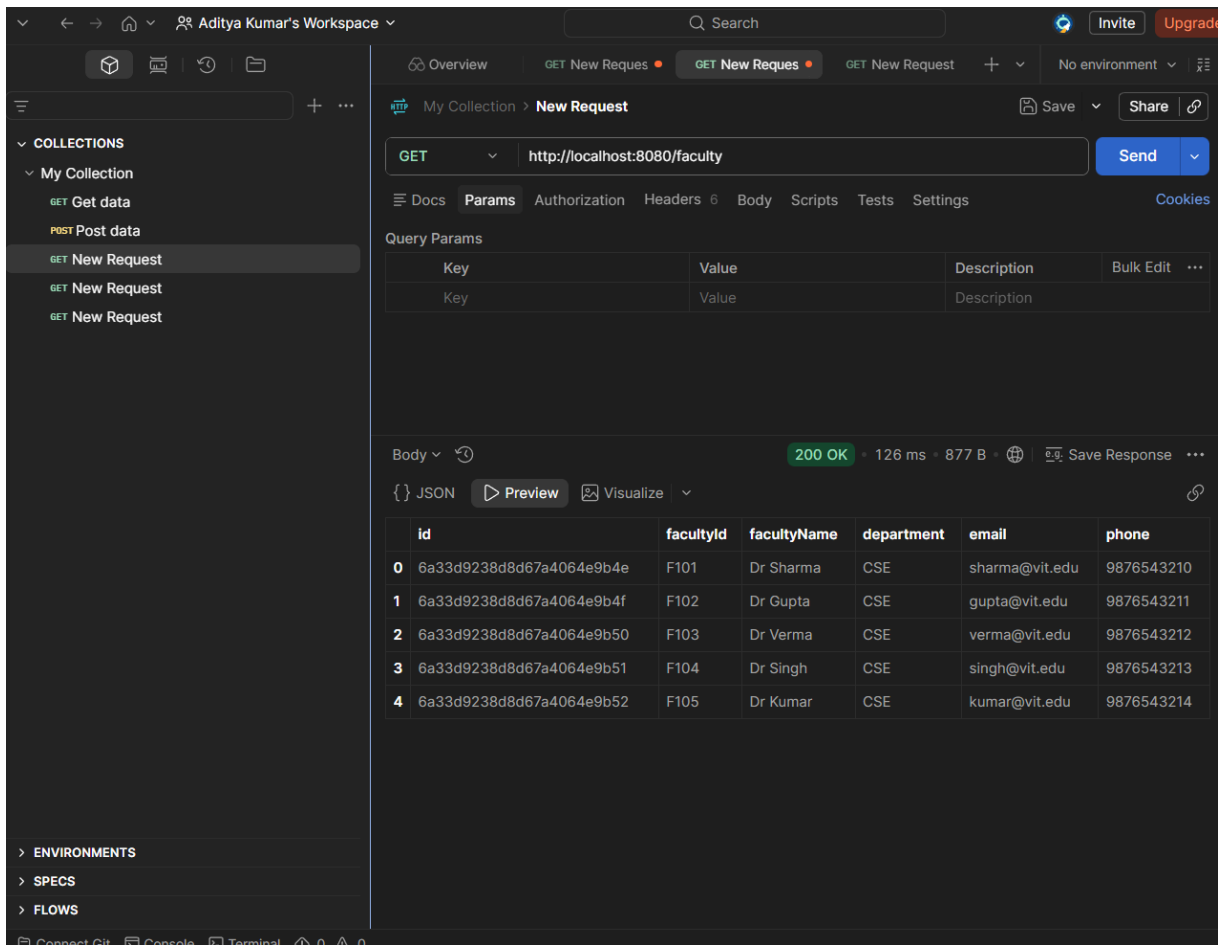


The screenshot displays a REST client interface with a GET request to `http://localhost:8080/courses`. The response is a 200 OK status with a response time of 1.96 s and a body size of 859 B. The response body is a JSON array of course data.

	id	courseId	courseName	credits	semester	facultyName
0	6a33ecf1932db15dec5d197c	C101	Database Management System	4	3	Dr Sharma
1	6a33ecf1932db15dec5d197d	C102	Object Oriented Programming	4	3	Dr Gupta
2	6a33ecf1932db15dec5d197e	C103	Operating Systems	4	4	Dr Verma
3	6a33ecf1932db15dec5d197f	C104	Computer Networks	3	4	Dr Singh
4	6a33ecf1932db15dec5d1980	C105	Java Programming	4	3	Dr Kumar

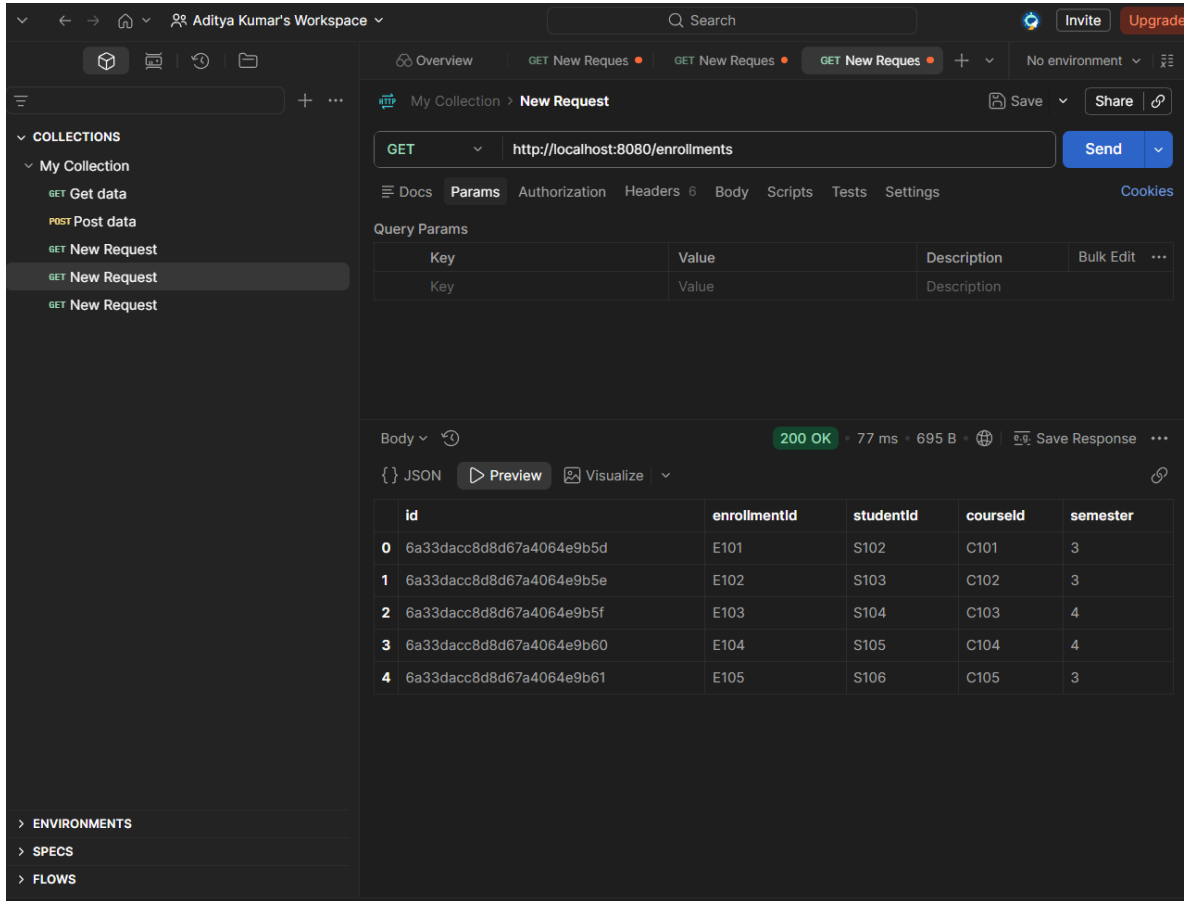
Faculty Module

The Faculty Module manages faculty-related information including faculty ID, name, department, email, and contact details. This helps maintain an organized faculty database.



Enrollment Module

The Enrollment Module establishes the relationship between students and courses. It records which student is enrolled in which course and semester.



The screenshot displays a REST client interface for "Aditya Kumar's Workspace". The active collection is "My Collection" with the selected item "GET New Request". The request is a GET to "http://localhost:8080/enrollments". The response is a 200 OK status with a 77 ms response time and 695 B of data. The response body is a JSON array of 5 enrollment records.

	id	enrollmentId	studentId	courseId	semester
0	6a33dacc8d8d67a4064e9b5d	E101	S102	C101	3
1	6a33dacc8d8d67a4064e9b5e	E102	S103	C102	3
2	6a33dacc8d8d67a4064e9b5f	E103	S104	C103	4
3	6a33dacc8d8d67a4064e9b60	E104	S105	C104	4
4	6a33dacc8d8d67a4064e9b61	E105	S106	C105	3

Marks Module

The Marks Module stores academic performance data of students. It records internal marks, external marks, and total marks for different subjects.

Aditya Kumar's Workspace

Search

Invite Upgrade

OverviewGET New Reql...GET New Reql...GET New Reql...GET http://loc...

SaveShare

My Collection

GET Get data

POST Post data

GET New Request

GET New Request

GET New Request

ENVIRONMENTS

SPECS

FLOWs

Connect GitConsoleTerminal00

http://localhost:8080/marks

GEThttp://localhost:8080/marksSend

DocsParamsAuthorizationHeaders6BodyScriptsTestsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit	
Key	Value	Description		

Body200 OK140 ms891 B

JSONPreviewVisualize

	id	markId	studentId	subject	internalMarks	externalMarks	totalMarks
0	6a33daf88d8d67a4064e9b62	M101	S102	DBMS	28	62	90
1	6a33daf88d8d67a4064e9b63	M102	S103	OOPS	25	60	85
2	6a33daf88d8d67a4064e9b64	M103	S104	OS	27	58	85
3	6a33daf88d8d67a4064e9b65	M104	S105	CN	29	64	93
4	6a33daf88d8d67a4064e9b66	M105	S106	JAVA	30	65	95

Attendance Module

The Attendance Module maintains attendance records for students. It tracks attendance status, subject information, and attendance dates for academic monitoring.

COLLECTIONS

My Collection

GET Get data

POST Post data

GET New Request

GET New Request

GET New Request

ENVIRONMENTS

SPECS

FLOWS

HTTP

http://localhost:8080/attendance

Save

Share

GET

http://localhost:8080/attendance

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Tests

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

200 OK

904 ms

815 B

...

{ } JSON

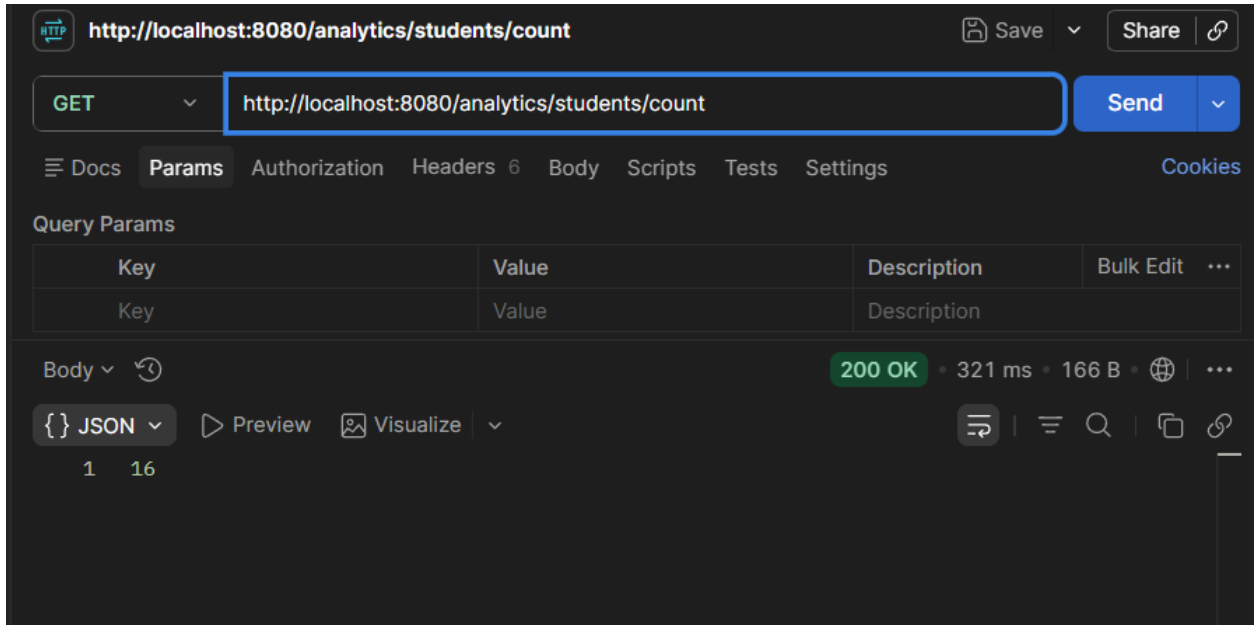
Preview

Visualize

...

	id	attendanceId	studentId	subject	date	status
0	6a33db4b8d8d67a4064e9b67	A101	S102	DBMS	2026-06-15	Present
1	6a33db4b8d8d67a4064e9b68	A102	S103	OOPS	2026-06-15	Present
2	6a33db4b8d8d67a4064e9b69	A103	S104	OS	2026-06-15	Absent
3	6a33db4b8d8d67a4064e9b6a	A104	S105	CN	2026-06-15	Present
4	6a33db4b8d8d67a4064e9b6b	A105	S106	JAVA	2026-06-15	Present

Analytics



HTTP **http://localhost:8080/analytics/students/count** Save Share

GET **http://localhost:8080/analytics/students/count** Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

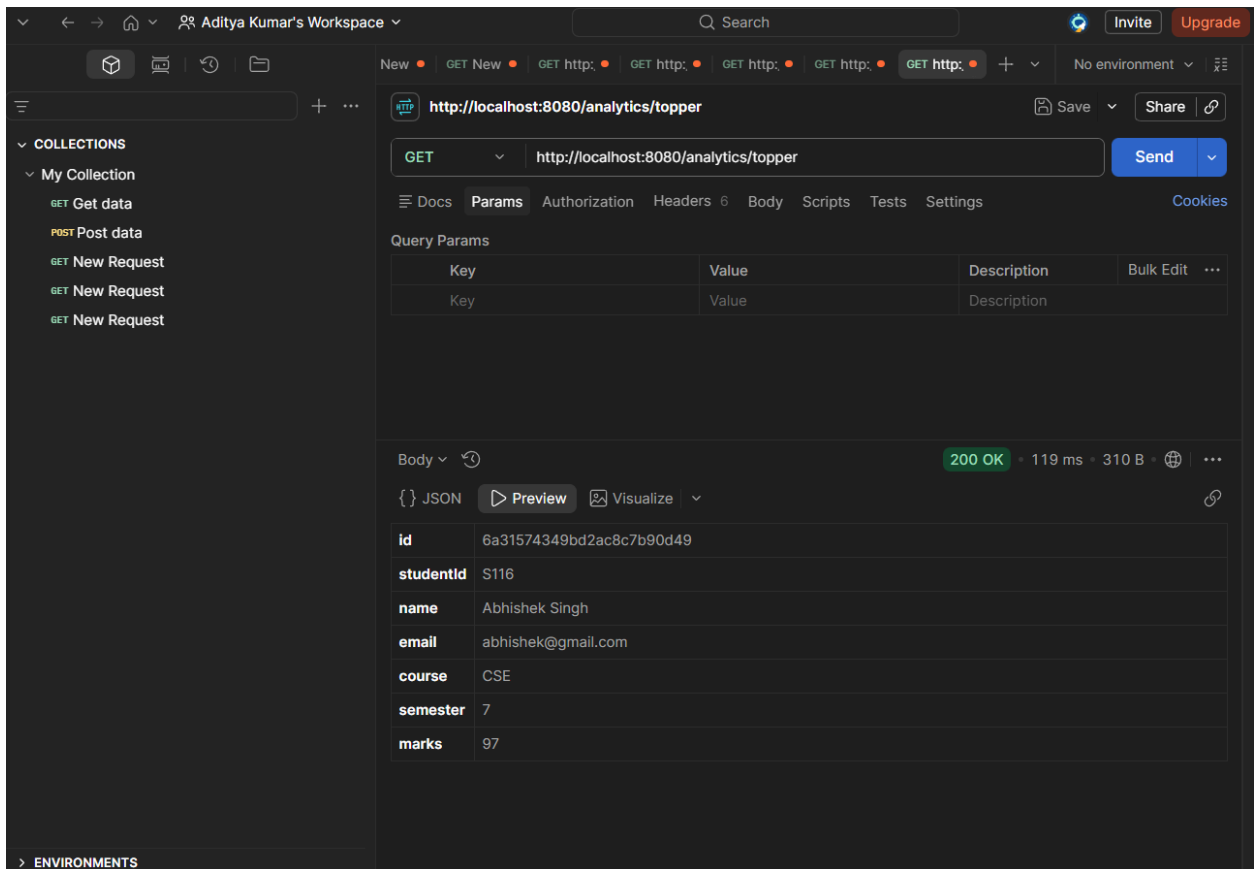
Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 321 ms • 166 B • ...

{ } JSON Preview Visualize

1 16

topper?



Aditya Kumar's Workspace Search Invite Upgrade

New GET New GET http: GET http: GET http: GET http: GET http: + No environment

HTTP **http://localhost:8080/analytics/topper** Save Share

GET **http://localhost:8080/analytics/topper** Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 119 ms • 310 B • ...

{ } JSON Preview Visualize

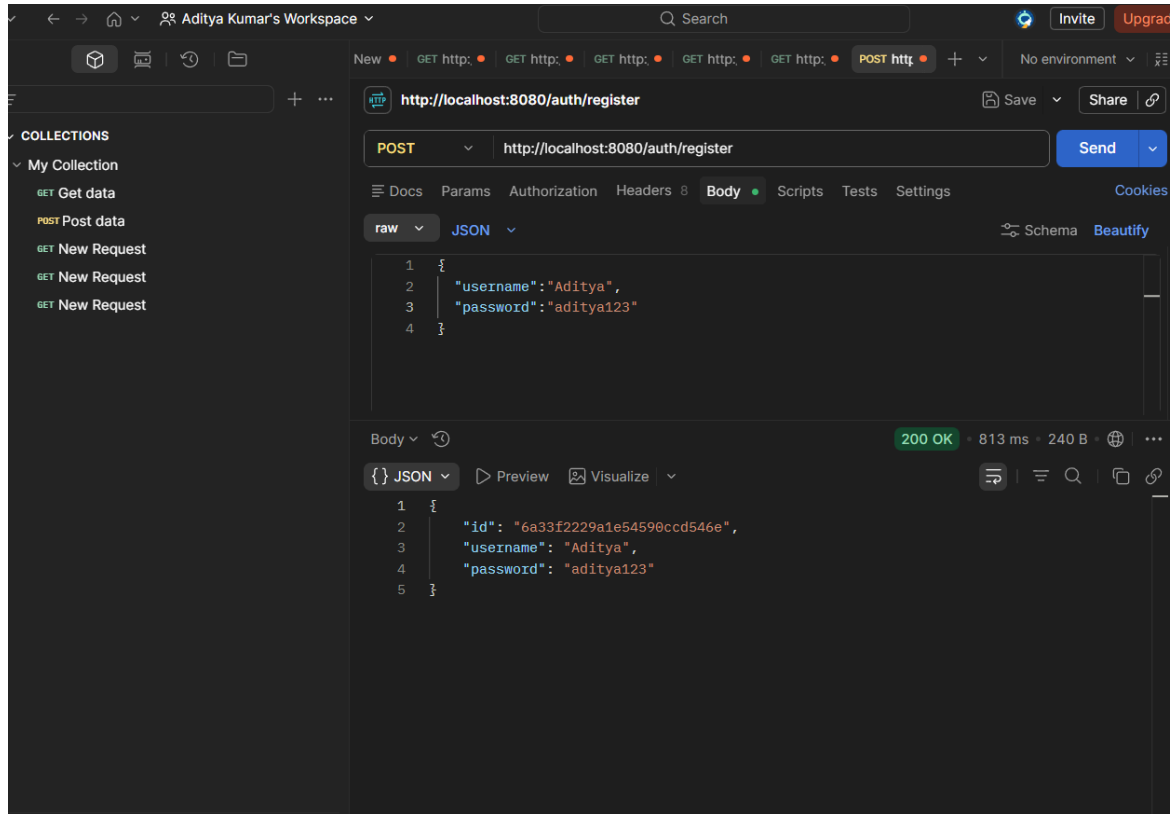
id	6a31574349bd2ac8c7b90d49
studentid	S116
name	Abhishek Singh
email	abhishek@gmail.com
course	CSE
semester	7
marks	97

> ENVIRONMENTS

Authentication

The Authentication Module provides a basic login mechanism for system access. User credentials are verified before granting access, enhancing application security.

Register



Login

Aditya Kumar's Workspace

Search

InviteUpgrad

http://localhost:3080/auth/login

SaveShare

POSThttp://localhost:3080/auth/login

Send

DocsParamsAuthorizationHeaders8BodyScriptsTestsSettings

Cookies

rawJSONSchemaBeautify

```
1 {
2   "username": "Aditya",
3   "password": "aditya123"
4 }
```

Body

200 OK • 219 ms • 180 B •

RawPreviewVisualize

1 Login Successful

Conclusion

The Student Management System was successfully developed using Spring Boot and MongoDB. The project demonstrates CRUD operations, REST API development, database connectivity, authentication, and data management features. The system provides an efficient way to manage students, courses, faculty, enrollments, marks, and attendance records. This project helped in understanding backend development concepts and NoSQL database integration in real-world applications.